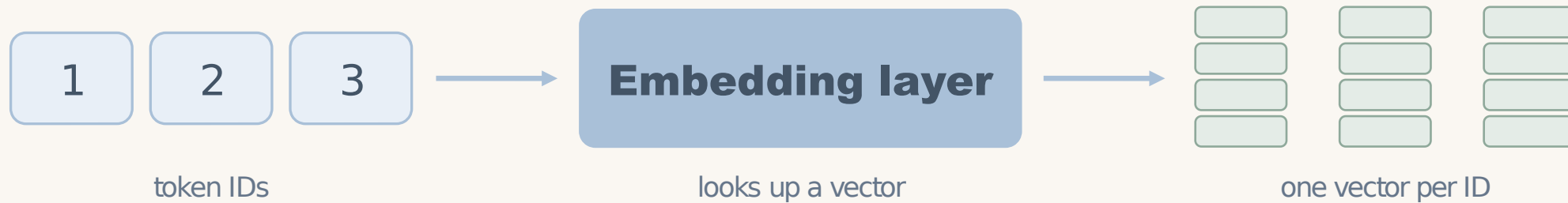


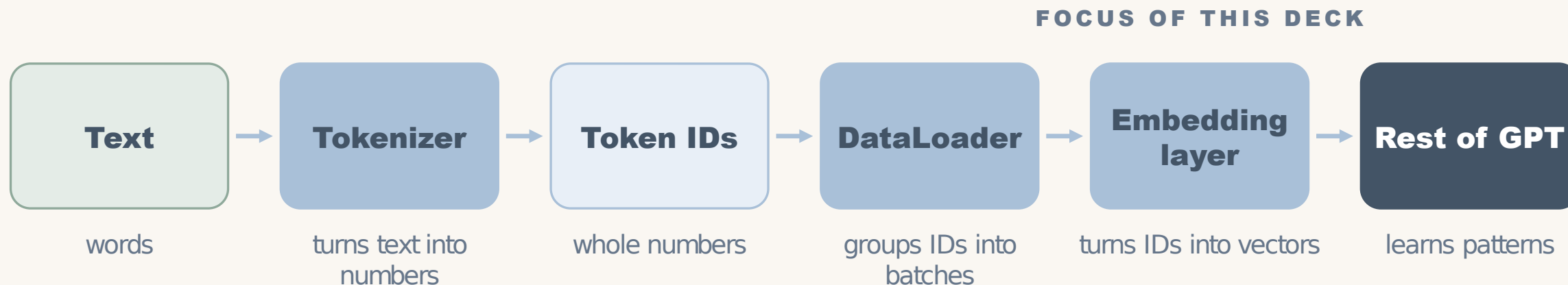
Embeddings: Giving Token IDs Meaning

How a GPT model turns plain numbers into something it can learn from



Where embeddings fit

Before a model can learn from text, the text goes through a few steps.



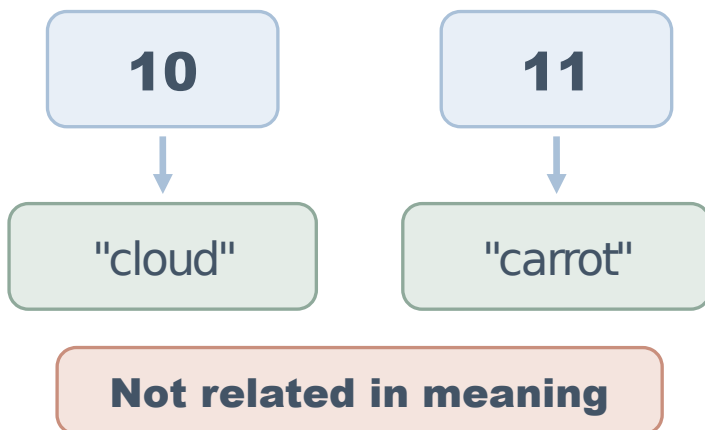
KEY IDEA

The embedding layer is the first step inside the model.

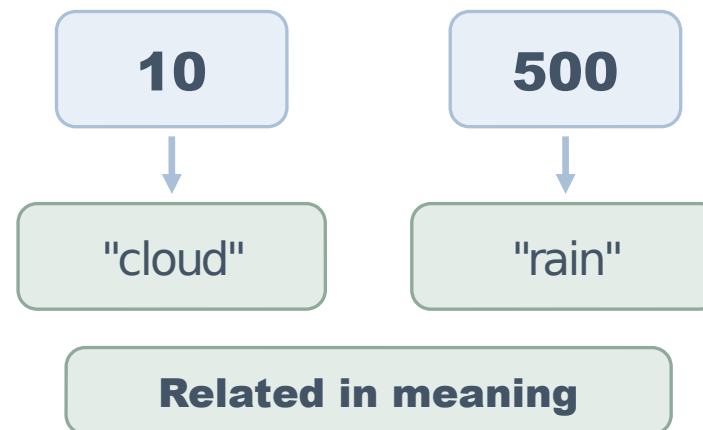
A token ID is just a label

A token ID is a whole number that points to one token in the vocabulary. (Example IDs below.)

Close numbers



Far-apart numbers



KEY IDEA

An ID is a name tag. The number says nothing about meaning.

Embeddings: a list of numbers per token

A vector is a list of numbers. An embedding is the vector the model keeps for one token.

token	ID	embedding (vector)						
"cloud"	10	→	0.8	0.1	-0.6	0.5	0.3	similar
"rain"	500	→	0.7	0.2	-0.5	0.6	0.2	
"carrot"	11	→	-0.4	0.9	0.3	-0.7	0.8	different

Example numbers. Real embeddings have hundreds of numbers per token.



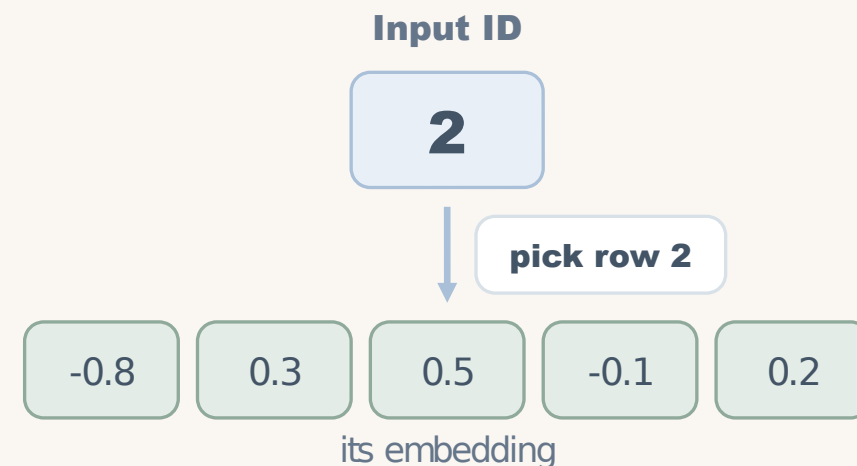
KEY IDEA

After training, tokens with similar meanings get similar vectors.

The token embedding table

The token table has one row per token ID. Looking up an ID means picking its row.

ID	vector (embedding_dim numbers)				
0	0.1	-0.3	0.9	0.2	-0.5
1	0.4	0.7	-0.2	0.0	0.6
2	-0.8	0.3	0.5	-0.1	0.2
3	0.2	-0.6	0.1	0.8	-0.4
4	0.5	0.0	-0.7	0.3	0.1
⋮	⋮				
last	0.3	0.2	-0.9	0.4	0.0



vocab_size: number of rows

embedding_dim: numbers in each row

Example, GPT-2 small: 50,257 rows × 768 numbers

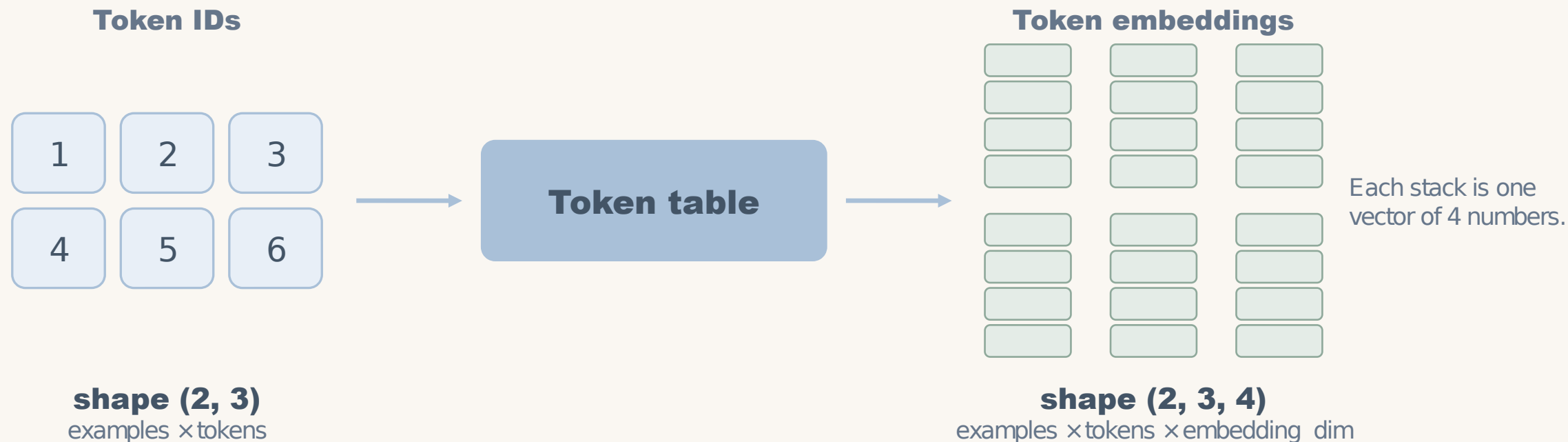


KEY IDEA

Looking up an embedding means picking a row by its ID.

Token embeddings for a whole batch

Every ID in the batch is swapped for its vector. The shape gains one new dimension.



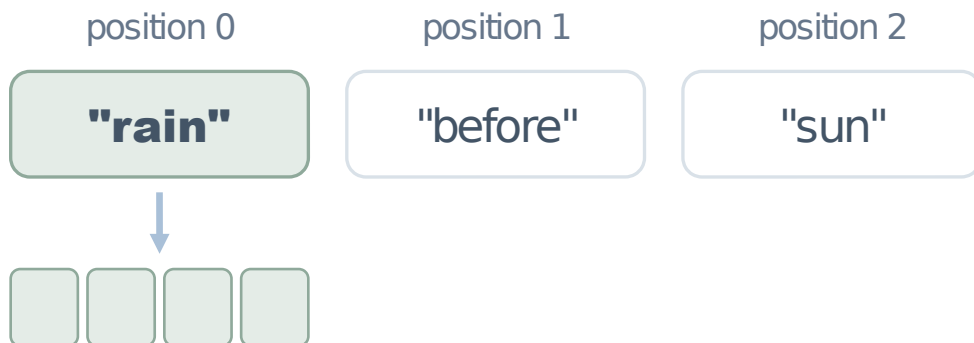
KEY IDEA

Each ID becomes a vector, so the shape gets one more number at the end.

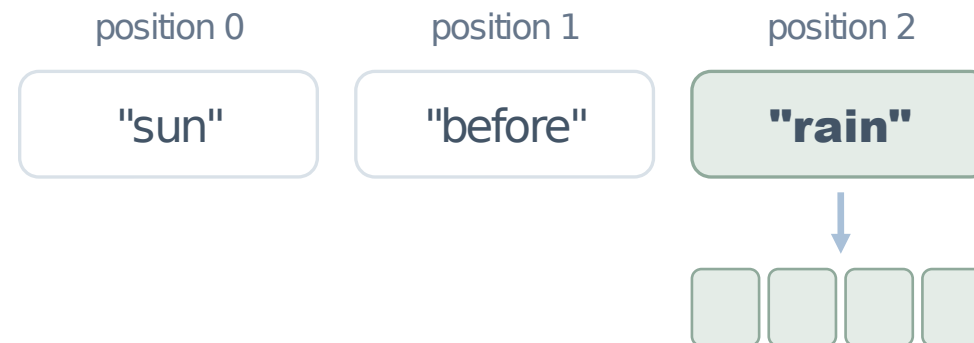
Why position matters

Word order changes meaning. But the token table ignores order.

rain before sun



sun before rain



"rain" gets the same token vector in both sentences.

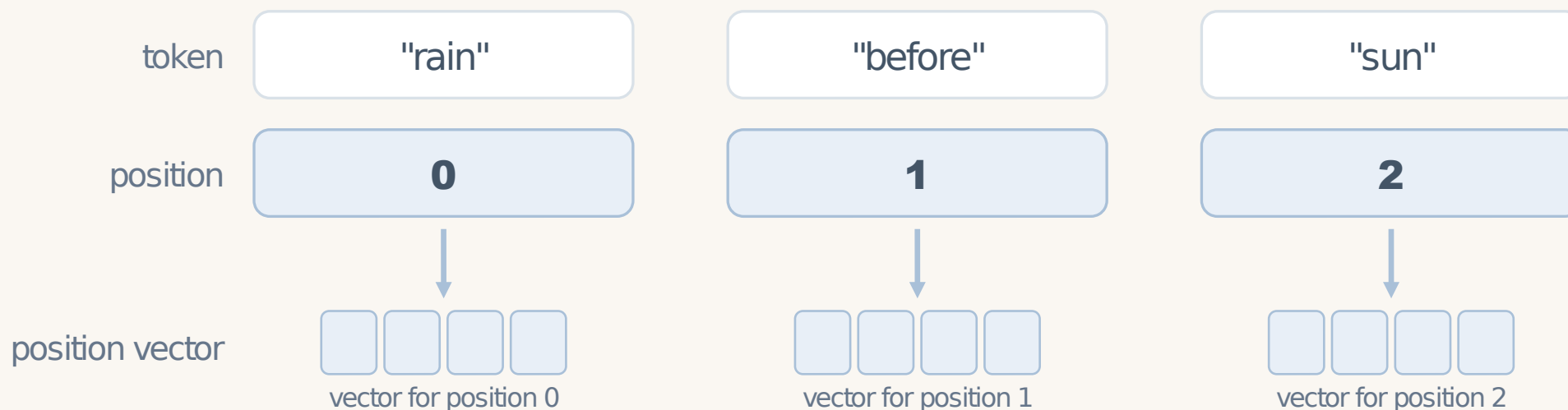


KEY IDEA

A token vector says what a token is, not where it is.

Position vectors

A second table gives every position its own vector.



Counted automatically: 0, 1, 2 ... up to the longest input allowed.

Absolute: position 2 always gets the same vector, whatever token sits there.



KEY IDEA

A position vector says where a token is.

Adding them together

Both vectors are the same length, so they are added number by number.

Example: "rain" at position 2

token vector what: "rain"	0.7	0.2	-0.5	0.4
+ position vector where: position 2	0.1	-0.3	0.2	0.0
= input vector what + where	0.8	-0.1	-0.3	0.4

"rain" at position 0 would add a different position vector, so it gets a different result.

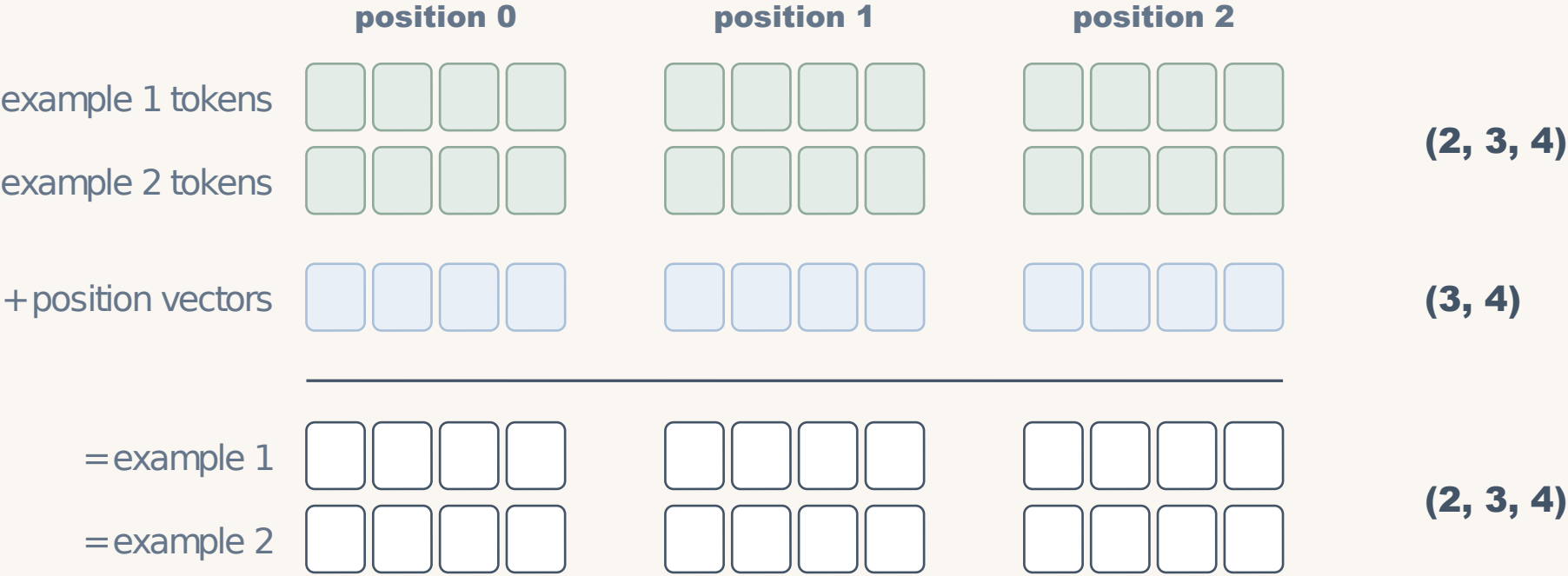


KEY IDEA

The input vector carries both what the token is and where it is.

Doing it for a whole batch

Each position vector is added to the token in that position, in every example.



KEY IDEA

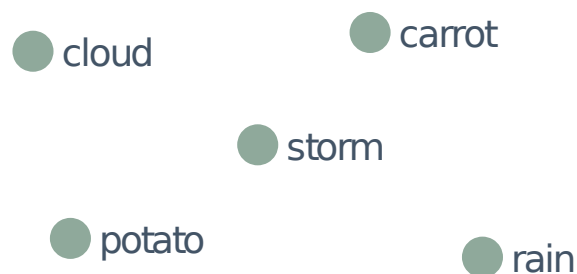
The shape stays (batch_size, sequence_length, embedding_dim).

The vectors are learned

Both tables start as random numbers. Each training step nudges them a little.

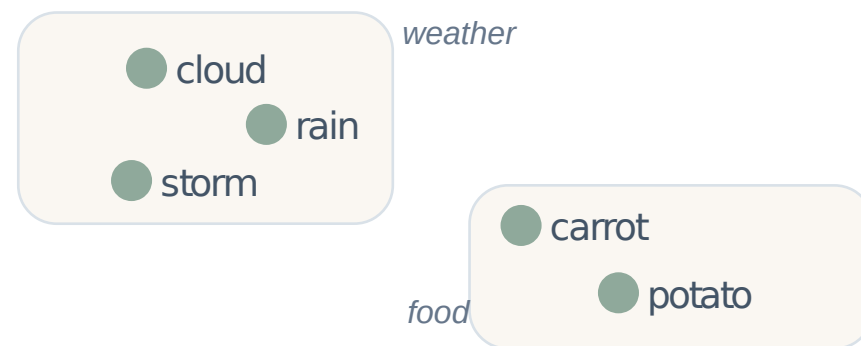
Before training

Random: no pattern yet



After training

Similar tokens sit close together



A 2-D sketch. Real embeddings have hundreds of numbers each.



TAKEAWAY

Token vectors say what. Position vectors say where. Adding them gives the model both.